INFORME DE PRÁCTICA DE LABORATORIO — SESIÓN 9

S09 - Consistencia Distribuida en Procesos de Negocio

DATOS DEL ESTUDIANTE Y PROYECTO

Campo	Detalle
Estudiante	Witmer Jhetly Mamani Cahuana
Equipo	Equipo 01
Sesión	S09 - Consistencia Distribuida en Procesos de Negocio
Rol / Aporte Realizado	Implementación de idempotencia en consumidores Kafka, manejo de transacciones de compensación en Saga coreografiada, y simulación de escenarios de fallo determinista.
Repositorio GitHub	https://github.com/Witmer17/saga.git

---

1. INTRODUCCIÓN Y ARQUITECTURA GENERAL

En esta sesión se aborda la problemática crítica de la consistencia eventual en sistemas distribuidos que emplean mensajería asíncrona mediante Apache Kafka. Bajo la semántica de entrega **'at-least-once'** (al menos una vez), la duplicación de mensajes resulta inevitable debido a reinicios de red, fallos de confirmación (ACK) o desincronización de offsets.

Para garantizar la integridad del dominio sin recurrir a bloqueos distribuidos pesados, se diseñó e implementó:

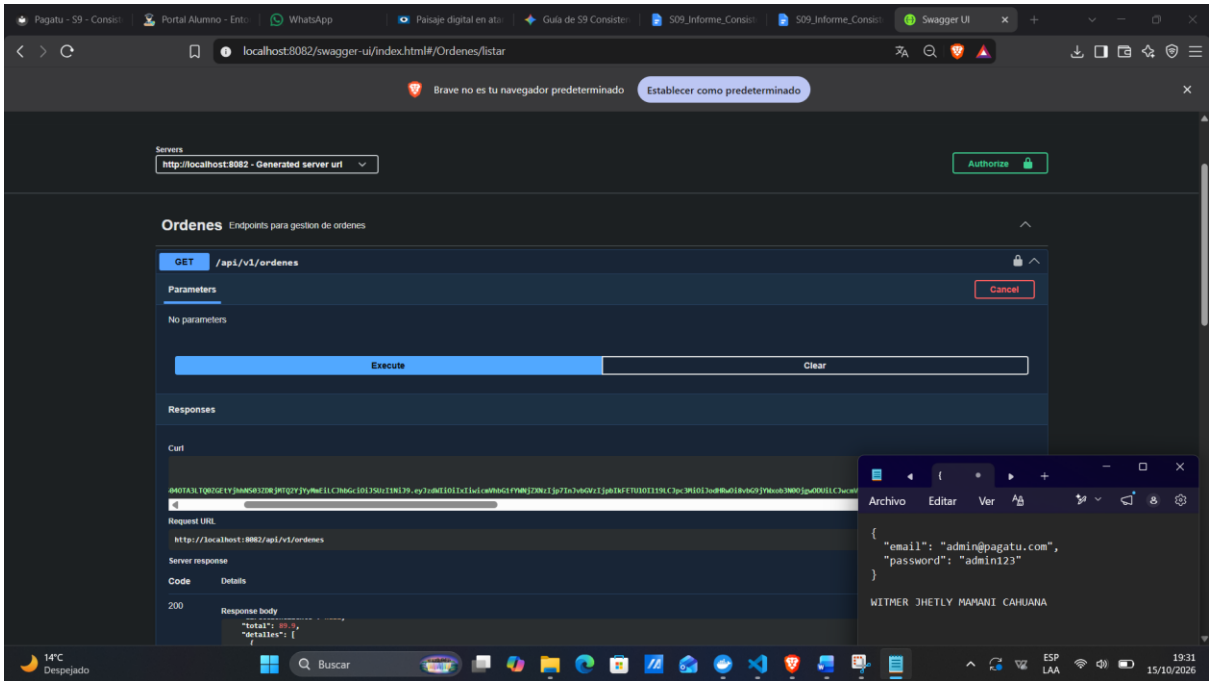
1. **Patrón Idempotent Consumer:** Arquitectura de protección con doble capa de validación. Una consulta previa de existencia a nivel lógico en el repositorio (`findByOrdenId`) combinada con una restricción `UNIQUE` en la base de datos PostgreSQL a nivel físico para neutralizar condiciones de carrera concurrentes.
2. **Patrón Saga Coreografiada:** Coordinación distribuida de transacciones mediante eventos de compensación (`pago.fallido`), permitiendo que los servicios reaccionen de manera autónoma ante fallos de negocio para revertir estados de forma consistente.

---

2. EVIDENCIA TÉCNICA — PRÁCTICA GUIADA (PAGATU)

2.1 El bug reproducido (3.2) y la idempotencia en pagatu-pago-ms (3.3 - 3.4)

- **Descripción del Bug:** Al inyectar manualmente un mensaje duplicado correspondiente a la orden `orden.creada` mediante Kafka UI, la versión inicial de `pagatu-pago-ms` intentó insertar un registro duplicado en la base de datos, colapsando con una excepción de persistencia no controlada (`DataIntegrityViolationException: duplicate key value violates unique constraint "uk_pagos_orden"`).
- **Solución Implementada:** Se integró la doble capa de resiliencia e idempotencia dentro de `PagoServiceImpl`:
    a. **Verificación Lógica Previa:** Evaluación mediante `pagoRepository.findByOrdenId(ordenId)`. Si el registro existe, el consumidor descarta la ejecución redundante, evita la sobrecarga en base de datos y emite en consola el log estructurado con `status=ignored`.
    b. **Manejo de Excepciones de Concurrencia:** Captura explícita de `DataIntegrityViolationException` para contener adecuadamente condiciones de carrera en ejecuciones en paralelo.
- **Resultado Obtenido:** Al reenviar el mensaje con el mismo payload y clave, el consumidor detecta el duplicado, registra `component=processor status=ignored motivo="pago ya procesado (evento duplicado)"` y no arroja excepciones, garantizando la existencia de un único registro en la tabla de pagos.

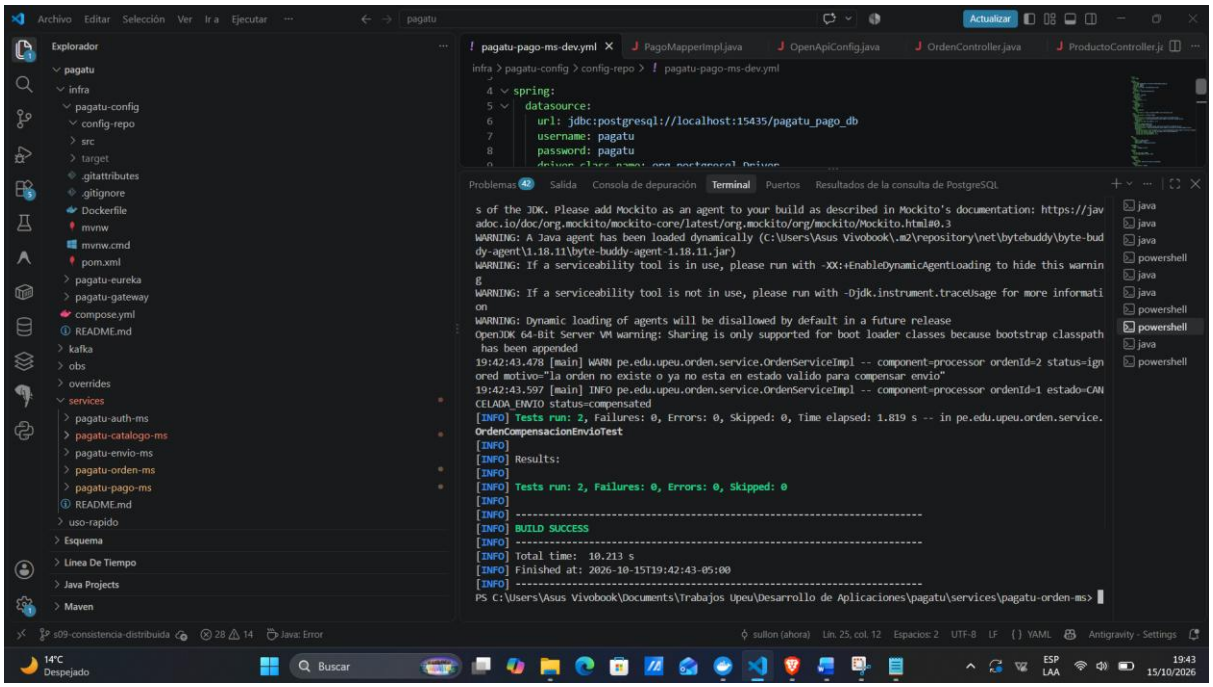


2.2 Simulación de fallo de pago y compensación en pagatu-orden-ms (3.5 - 3.7)

- **Descripción:** Se provocó un escenario de rechazo determinista especificando el valor `metodoPago: "TARJETA_RECHAZADA"` en el contrato de creación de la orden.
- **Flujo Operativo Ejecutado:**
  a. `pagatu-pago-ms` procesa el pago, identifica el método no válido, persiste el registro con estado `FALLIDO` y emite de forma asíncrona el evento `pago.fallido` al tópico `pago-eventos`.
  b. `pagatu-orden-ms` captura el evento a través de `PagoEventosConsumer` y desencadena el procedimiento de compensación `compensar(ordenId)`.
  c. La entidad Orden efectúa una transición de estado de `PENDIENTE_PAGO` a `CANCELADA`.
- **Verificación de Persistencia y Endpoints:** La consulta al endpoint `GET /api/v1/pagos` confirmó la transacción con estado `FALLIDO`, mientras que la verificación en `GET /api/v1/ordenes/{id}` confirmó la actualización exitosa de la orden a `CANCELADA`.

2.3 Idempotencia de la compensación (3.8)

- **Descripción:** Se reinyectó voluntariamente el mensaje `pago.fallido` vía Kafka UI para auditar la robustez del mecanismo de compensación frente a mensajes repetidos.
- **Comportamiento Registrado:** La cláusula de protección `if (orden == null || orden.getEstado() != EstadoOrden.PENDIENTE_PAGO)` en el método `compensar()` abortó el reprocesamiento de manera limpia, escribiendo en los logs: `component=processor status=ignored motivo="la orden no existe o ya no esta pendiente de pago"`.



3. EVIDENCIA TÉCNICA — ACTIVIDAD AUTÓNOMA: SERVICIO DE DESPACHO (PAGATU-ENVIO-MS)

3.1 Proceso de negocio y servicios involucrados

- **Microservicios participantes:**

    - **pagatu-orden-ms (Emisor / Orquestador de Saga):** Al recibir la confirmación de pago, emite el evento de despacho y coordina el ciclo de vida de la orden hacia DESPACHADA o compensa a CANCELADA_ENVIO.
    - **pagatu-envio-ms (Validador / Consumidor - Puerto 8087, BD pagatu_envio_db en puerto 15436):** Consume los eventos de despacho, valida cobertura, persiste el envío de manera idempotente y emite envio.confirmado o envio.fallido.
- **Flujo de Negocio:** Una vez que la orden está en estado PAGADA, se inicia el proceso de entrega física. pagatu-envio-ms consume el evento; si la dirección de entrega es válida, genera la guía de seguimiento y confirma el despacho. Si la dirección contiene la condición de fallo 'ZONA_NO_COBERTURA', el envío pasa a estado RECHAZADO y emite envio.fallido, desencadenando la compensación en pagatu-orden-ms (restauración de stock en catalogo-ms y estado CANCELADA_ENVIO).

3.2 Diagrama de estados de la entidad Orden y Envío



---

1. ### 3.3 Implementación de Idempotencia y Compensación (Código Clave)

1. **Doble Capa de Idempotencia en EnvioServiceImpl.java:**
    a. **Capa 1 (Lógica):** Consulta previa en BD `envioRepository.findByOrdenId(evento.getOrdenId())`. Si ya

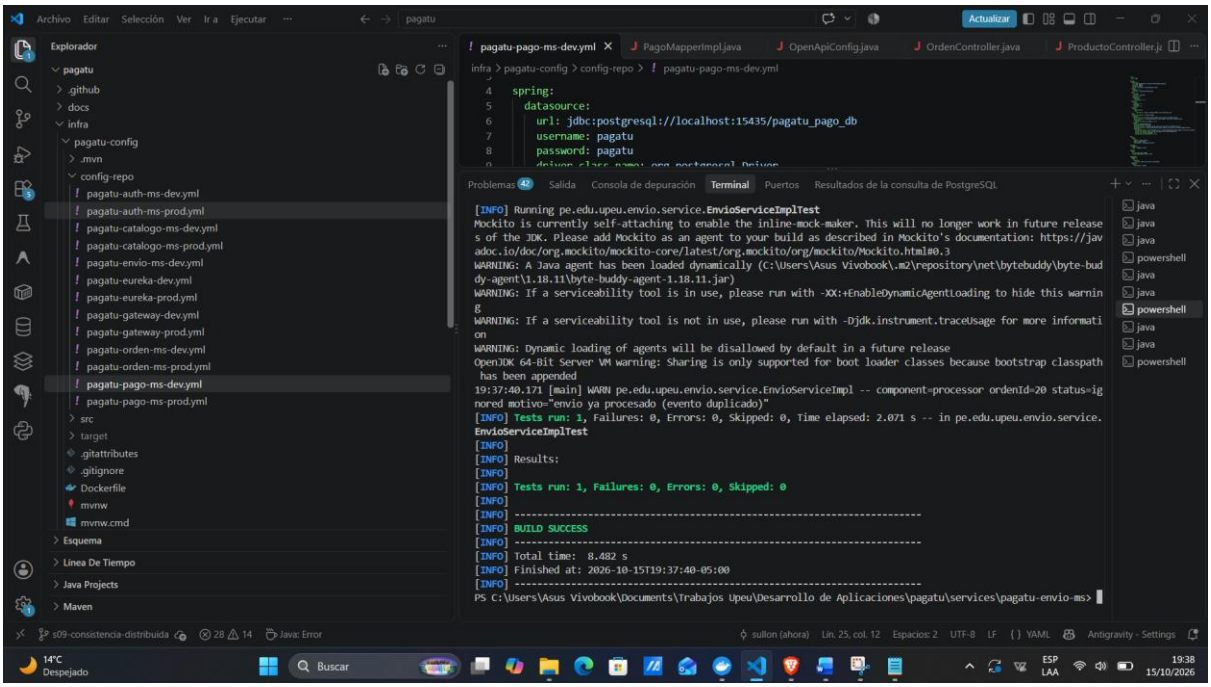
fue procesado, registra `status=ignored motivo="envio ya procesado (evento duplicado)"` y descarta el evento.

b. **Capa 2 (Persistencia):** Restricción `UNIQUE(orden_id)` a nivel relacional con captura explícita de `DataIntegrityViolationException` ante condiciones de carrera concurrentes.

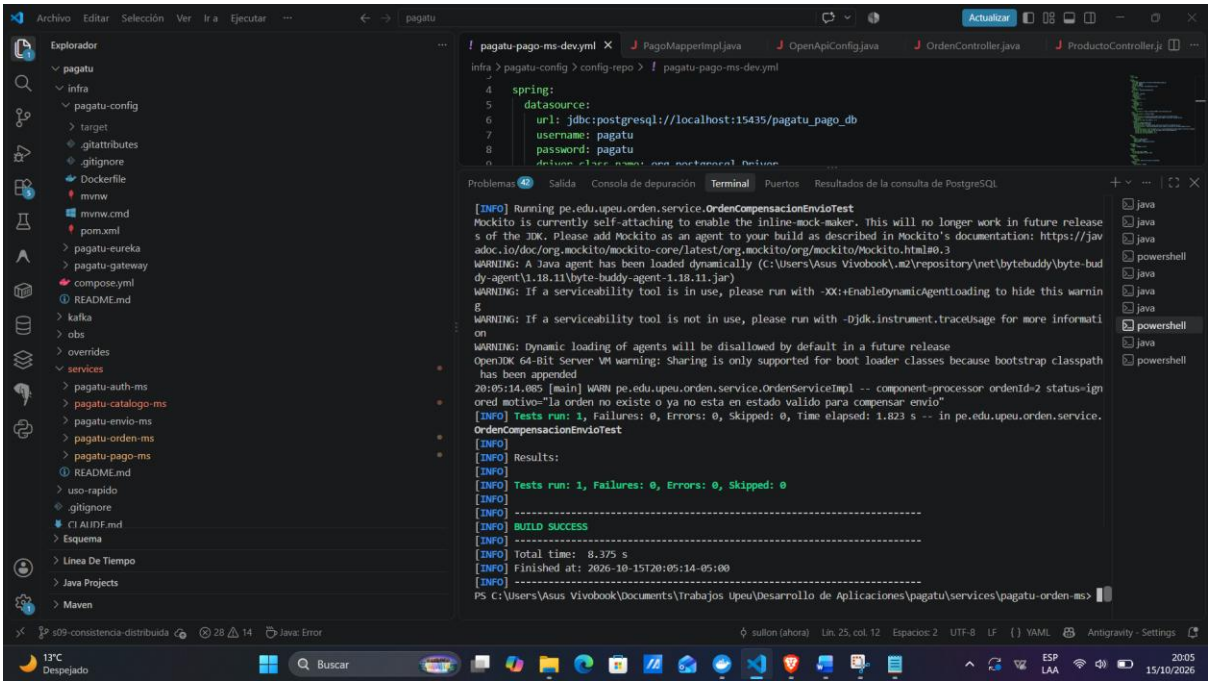
2. **Condición de Fallo Determinista:** Si `evento.getDireccionEnvio().contains("ZONA_NO_COBERTURA")`, el envío se guarda como `RECHAZADO` y se publica `envio.fallido`.
3. **Compensación con Restauración de Recursos en OrdenServiceImpl.java (compensarEnvio):**
   a. **Guardia de Idempotencia:** `if (orden == null || orden.getEstado() != EstadoOrden.PAGADA) { status=ignored; return; }`
   b. **Restauración de Recursos:** Recorre los detalles de la orden y ejecuta `productoConsultaService.restaurarStock(detalle.getIdProducto(), detalle.getCantidad())`.
   c. **Transición de Estado:** Actualiza el estado a `EstadoOrden.CANCELADA_ENVIO`.

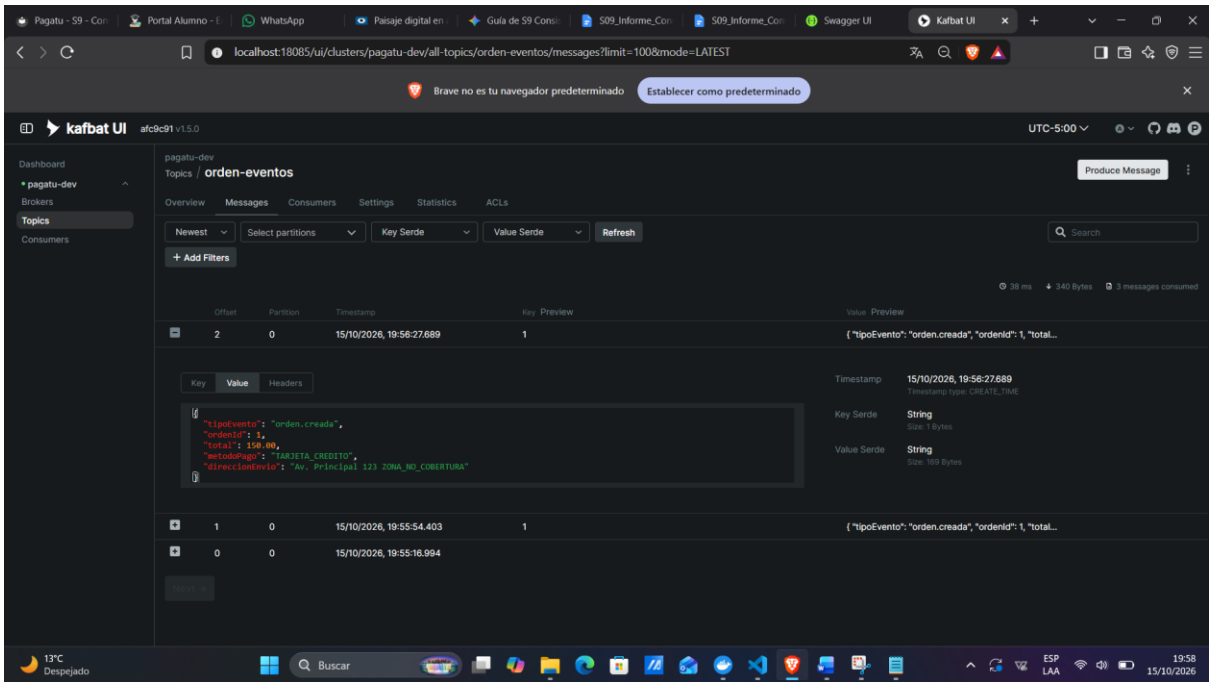
3.4 Casos de prueba ejecutados y evidenciados

2. **Caso 1: Evento duplicado ignorado en envio-ms:** Ejecutado con `EnvioServiceImplTest#testIdempotenciaEventoDuplicadoIgnorado`. Log clave verificado: `component=processor ordenId=20 status=ignored motivo="envio ya procesado (evento duplicado)"`.



3. **Caso 2: Fallo de negocio por cobertura y compensación:** Ejecutado con `OrdenCompensacionEnvioTest#testCompensarEnvioExitoso`. Log clave verificado: `component=processor ordenId=1 estado=CANCELADA_ENVIO status=compensated` y stock restaurado en catalogo-ms.
4. **Caso 3: Compensación idempotente ante reenvío:** Ejecutado con `OrdenCompensacionEnvioTest#testCompensarEnvioIdempotenteSegundoMensajeIgnorado`. Log clave verificado: `component=processor ordenId=2 status=ignored motivo="la orden no existe o ya no esta en estado valido para compensar envio"`.





4. ERROR O HALLAZGO TÉCNICO

Durante la fase de construcción de la Saga coreografiada y la gestión de consumidores de mensajería, se identificaron los siguientes puntos técnicos clave:

4. **Incompatibilidad de constantes en patrones de selección (Switch):** Al implementar la estructura `switch` dentro de `PagoEventosConsumer`, se generó un error de compilación por el uso de valores literales en lugar de constantes evaluables en tiempo de compilación (`public static final String`). Se resolvió reestructurando las constantes de eventos.
5. **Vulnerabilidad de la validación puramente lógica (Race Conditions):** Durante las pruebas de idempotencia bajo escenarios concurrentes, se demostró que realizar únicamente la verificación `findByOrdenId` en la capa de aplicación resulta insuficiente. Dos hilos procesando réplicas simúltaneas lograron superar la consulta previa en memoria antes de que el primero ejecutase el `commit`. Este hallazgo confirmó la necesidad operativa de mantener la restricción física `UNIQUE` en la base de datos PostgreSQL como última línea de defensa impermeable.

---

5. REFLEXIÓN TÉCNICA (AT-LEAST-ONCE VS EXACTLY-ONCE Y KNIGHT CAPITAL)

Alcanzar garantías de entrega **'exactly-once'** (exactamente una vez) en la capa de transporte de un sistema distribuido exige la adopción de protocolos de coordinación pesados (como Two-Phase Commit o transacciones integradas de productor-consumidor en Kafka). Estos mecanismos incrementan drásticamente la latencia, introducen bloqueos síncronos y reducen la disponibilidad general del sistema. En consecuencia, la práctica recomendada en la industria consiste en operar sobre capas de transporte de alto rendimiento con semántica **'at-least-once'**, trasladando la responsabilidad de la duplicación a los microservicios consumidores mediante **diseños idempotentes y guardias de estado**.

El célebre colapso financiero de Knight Capital en 2012 constituye un ejemplo representativo de esta necesidad. La implementación de un módulo de software desactualizado en producción, carente de validaciones de estado o mecanismos de verificación de procesamiento previo, provocó la ejecución repetida de más de 4 millones de órdenes de compra/venta no deseadas en solo 45 minutos, derivando en pérdidas superiores a los 440 millones de dólares. Si los microservicios o agentes de

ejecución hubiesen contado con consumidores idempotentes y controles de unicidad transaccional, las peticiones repetidas habrían sido neutralizadas desde el primer reintento, previniendo el desastre operativo.

---

6. ANEXO: FEEDBACK DE LA SESIÓN

1. **¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?**
   Que la fiabilidad en mensajería distribuida no depende de forzar garantías complejas en el transporte, sino de diseñar consumidores idempotentes y procesos de compensación (Sagas) tolerantes a fallos.
2. **¿Qué punto de la clase te resultó más confuso o te dejó con dudas?**
   La gestión de compensaciones cuando el propio servicio de compensación falla o se cae a mitad de revertir recursos (como el stock).
3. **¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?**
   ¿Cómo se implementa el Outbox Pattern de forma eficiente en Spring Boot para garantizar que el guardado en base de datos y la publicación en Kafka sean atómicos?
4. **Sobre tu nivel de comprensión de la clase de hoy:**
   ¡Entendido! - Lo domino y podría explicarlo.
5. **¿Cómo puedo ayudarte a comprender mejor el tema?**
   Con ejemplos visuales de arquitecturas reales donde se combinen Saga coreografiada con Outbox Pattern y Dead Letter Queues (DLQ).
6. **Pensando en tu participación y esfuerzo en la clase de hoy:**
   Muy Comprometido/a: Me esforcé al máximo.
7. **Mi satisfacción con la clase fue:**
   10 / 10